

CFGS Desenvolupament d'Aplicacions Multiplataforma (DAM) CFGS Desenvolupament d'Aplicacions Web (DAW)

Mòdul 0487 – Entorns de Desenvolupament. Unitat 2 – Optimització de programari

EAC3

(Curs 2025–26 / 1r semestre)

Enunciat

1. (1 punt) Responen les següents preguntes tipus test a la taula del final de l'exercici (als requadres blaus). L'exercici consta de 10 preguntes. Totes les preguntes compten igual (0,1 punts). Cada resposta equivocada resta 0,05 punts.

1.1. Qui realitza normalment les proves unitàries?

- A) El client final.
- B) Els analistes.
- C) Els programadors.
- D) Els verificadors externs.

1.2. Quin és l'objectiu principal de les proves d'acceptació?

- A) Validar la interfície gràfica.
- B) Verificar la velocitat del codi.
- C) Validar el producte amb els usuaris finals.
- D) Verificar la seguretat del sistema.

1.3. Quina tècnica s'utilitza per analitzar els camins d'execució interns d'un programa?

- A) Capsa blanca.
- B) Capsa negra.
- C) Capsa grisa.
- D) Capsa tancada.

1.4. Quin document defineix els continguts del pla de proves segons la normativa IEEE?

- A) IEEE 802.11
- B) IEEE 829-2008
- C) ISO 27001
- D) W3C 1.1

1.5. En les proves de capsa negra, el focus principal és:

- A) El flux de control intern.
- B) Els camins independents.
- C) Les entrades i sortides del sistema.
- D) Cap de les anteriors.

1.6. La complexitat ciclomàtica mesura:

- A) El nombre de línies del codi.
- B) El nombre de funcions dins d'una classe.
- C) El temps d'execució mitjà de totes les opcions d'un programa.
- D) El nombre de camions independents d'un programa.

1.7. Quina és la finalitat de la refacció?

- A) Canviar la funcionalitat d'un programa.
- B) Afegir noves funcionalitats.
- C) Millorar la qualitat interna del codi sense alterar-ne el comportament.
- D) Traduir el codi a un altre llenguatge.

1.8. Quina és una eina típica de control de versions?

- A) Eclipse.
- B) Git.
- C) JUnit.
- D) Jenkins.

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 1 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

1.9. Quin fitxer descriu l'estructura dels comentaris JavaDoc?

- A) fitxer .java amb etiquetes /** */
- B) javadoc.xml
- C) readme.md
- D) documentació HTML generada.

1.10. Quina afirmació és certa?

- A) Les proves són opcionals en petits projectes o quan hi hagi poc personal involucrat.
- B) Com més aviat es detecti un error, més barat és corregir-lo.
- C) Les proves no afecten la qualitat percebuda.
- D) Totes les anterior.

RESPOSTES

1.1	1.2	1.3	1.4	1.5	1.6	1.7	1.8	1.9	1.10
C	C	A	B	C	D	C	B	A	B

2. (3 punts) Sistemes de control de versions (Git).

Per realitzar aquest exercici caldrà que instal·leu el programa Git. Per fer-ho cal anar a aquesta pàgina: <https://git-scm.com/downloads>

Seguiu rigorosament els passos que s'indiquen a continuació. Al *prompt* ha d'aparèixer el camí de la carpeta actual. Al quadre del final, adjunteu una captura de pantalla de cadascuna (a les qüestions 2.6 i 2.14, que tenen una part de pregunta de resposta oberta, responeu-la).

2.0 Elaboració del codi a gestionar amb Git

A la vostra carpeta de treball, creeu el fitxer .java amb el codi que trobareu a continuació:

```
// Fitxer Robot.java

public class Robot {
    private String nom;
    private int bateria;

    public void setNom(String nom) {
        this.nom=nom;
    }
    public String getNom() {
        return nom;
    }

    public void setBateria(int bateria) {
        this.bateria=bateria;
    }

    public double getBateria() {
        return bateria;
    }
}
```

2.1 Configuració del repositori local

Inicialitzeu un nou projecte amb Git anomenat EAC3_Robot

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 2 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3>git init EAC3Robot
Initialized empty Git repository in D:/Users/Achraf/Documents/Instituto/Achraf/Entorns de desenvolupament/EAC3/EAC3Robot/.git/
```

2.2. Addició del fitxer al control de versions. (ordre git add)

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git add robot.java
```

2.3. Canvis pendents de confirmar.

Executeu l'ordre necessària perquè Git mostri els canvis pendents de confirmar (juntament amb altra informació) i enganxeu a continuació el resultat.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   robot.java
```

2.4. Confirmació dels canvis.

Confirmeu els canvis posant "primer versió de Cognom" com a missatge (substituïu *Cognom* pel vostre cognom). Torneu a executar l'ordre necessària perquè Git mostri els canvis pendents de confirmar i enganxeu el resultat.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git commit -m "primer versió de Ben Tamou"
[master (root-commit) be27d40] primer versió de Ben Tamou
 1 file changed, 22 insertions(+)
 create mode 100644 robot.java

D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git status
On branch master
nothing to commit, working tree clean
```

2.5. Creació de branques.

Creeu una nova branca anomenada branca1 i enganxeu a continuació el resultat.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git branch branca1
```

2.6. Mostra de branques.

Mostreu el llistat de branques que hi ha i digueu a quina esteu situats.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git branch
branca1
* master
```

La que tiene el * es en la cual estamos situados actualmente, en mi caso la master.

2.7. Moviment de branques.

Situeu-vos a la nova branca que heu creat i enganxeu a continuació el resultat.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git checkout branca1
Switched to branch 'branca1'
```

2.8. Actualització del codi a branca1.

Modifiqueu el fitxer Robot.java que heu creat abans perquè quedi de la següent manera:

```
// Fitxer Robot.java

public class Robot {
    private String nom;
    private int bateria;
    private String modeOperatiu;

    public void setNom(String nom) {
        this.nom=nom;
    }
    public String getNom() {
        return nom;
    }

    public void setBateria(int bateria) {
        this.bateria=bateria;
    }

    public double getBateria() {
        return bateria;
    }

    public void setModeOperatiu(String modeOperatiu) {
        this.modeOperatiu=modeOperatiu;
    }
    public String getModeOperatiu() {
        return modeOperatiu;
    }
}
```

2.9. Canvis pendents de confirmar.

Executeu l'ordre necessària perquè Git mostri els canvis pendents de confirmar i enganxeu el resultat.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git status
On branch branca1
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   robot.java

no changes added to commit (use "git add" and/or "git commit -a")
```

2.10. Afegir nous fitxers.

Afegiu novament el fitxer al control de versions (amb l'ordre **add**).

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git add robot.java
```

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 4 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

2.11. Confirmació dels canvis.

Confirmeu els canvis posant "segona versió de Cognom" com a missatge (a on Cognom serà el teu, per exemple «segona versió de Gimenez»).

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git commit -m "segunda versió de Ben Tamou"
[branca1 ade7d8d] segunda versió de Ben Tamou
1 file changed, 7 insertions(+)
```

2.12. Treball amb una segona branca (branca2).

Creeu la branca2, situeu-vos-hi i modifiqueu el fitxer Robot.java de la següent manera:

```
// Fitxer Robot.java

public class Robot {
    private String nom;
    private int bateria;
    private double velocitat;

    public void setNom(String nom) {
        this.nom=nom;
    }
    public String getNom() {
        return nom;
    }

    public void setBateria(int bateria) {
        this.bateria=bateria;
    }

    public double getBateria() {
        return bateria;
    }

    public void setVelocitat(double velocitat) {
        this.velocitat=velocitat;
    }
    public double getVelocitat() {
        return velocitat;
    }
}
```

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git branch branca2
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git checkout branca2
Switched to branch 'branca2'
```

Afegiu tots els fitxers al control de versions i feu un *commit* que tingui en compte aquests canvis amb el missatge "versió 3 de Cognom".

Eenganxeu una captura amb l'execució d'aquestes ordres i el seu resultat:

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git add robot.java
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git commit -m "versio 3 de Ben Tamou"
[branca2 dd7c2e5] versio 3 de Ben Tamou
1 file changed, 5 insertions(+), 5 deletions(-)
```

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 5 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

2.13. Diferències entre branques.

Mostra les diferències entre la branca actual (branca2) i la branca1

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git diff branca1 branca2
diff --git a/robot.java b/robot.java
index 23d64b2..2a08565 100644
--- a/robot.java
+++ b/robot.java
@@ -3,7 +3,7 @@
 public class Robot {
     private String nom;
     private int bateria;
-    private String modeOperatiu;
+    private double velocitat;

     public void setNom(String nom) {
         this.nom=nom;
@@ -20,10 +20,10 @@ public class Robot {
     }

     public void setModeOperatiu(String modeOperatiu) {
         this.modeOperatiu=modeOperatiu;
+    public void setVelocitat(double velocitat) {
+        this.velocitat=velocitat;
     }

     public String getModeOperatiu() {
         return modeOperatiu;
+    public double getVelocitat() {
+        return velocitat;
     }
 }
```

2.14. Fusió de branques.

Situat a la branca màster i fusiona la branca 1. Un cop fusionada la branca 1 fusiona la branca 2.

```
D:\Users\Achraf\Documents\Instituto\Achraf\Entorns de desenvolupament\EAC3\EAC3Robot>git merge branca2
Auto-merging robot.java
CONFLICT (content): Merge conflict in robot.java
Automatic merge failed; fix conflicts and then commit the result.
```

Què succeeix? Per què? Té solució?

Aparece el error de conflicto de fusión. Cuando usamos el comando para fusionar la branca2 pues entra en conflicto porque está modificando la misma parte de código pero con líneas diferentes. La solución sería abrirlo manualmente y editarlo quitando las líneas de nos dibuja git para mostrarnos que líneas son las que interfieren y así conseguiríamos tener los cambios de las dos ramas juntas.

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 6 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

3. (2,5 punts) Refacció.

En un sistema de comerç electrònic, cada client pot obtenir un descompte en funció del seu tipus de compte i de la quantitat total de la compra. Els descomptes s'apliquen de manera diferent segons si el client és bàsic, prèmium o vip.

Tenim la següent classe que calcula el preu final després d'aplicar el descompte corresponent:

```
package refaccio;

public class GestorDescomptes {

    public static double calculaPreuFinal(double totalCompra, String tipusClient) {
        double descompte = 0;
        double preuFinal = 0;

        if (tipusClient.equals("basic")) {
            if (totalCompra > 100) descompte = totalCompra * 0.05; //5% de descompte
            preuFinal = totalCompra - descompte;
            System.out.println("Client BASIC. Descompte aplicat: " + descompte + " €. Preu
                               final: " + preuFinal + " €.");
            return preuFinal;
        } else if (tipusClient.equals("premium")) {
            if (totalCompra > 100) descompte = totalCompra * 0.10; //10% de descompte
            if (totalCompra > 500) descompte = totalCompra * 0.15; //15% de descompte
            preuFinal = totalCompra - descompte;
            System.out.println("Client PREMIUM. Descompte aplicat: " + descompte + " €. Preu
                               final: " + preuFinal + " €.");
            return preuFinal;
        } else if (tipusClient.equals("vip")) {
            if (totalCompra > 200) descompte = totalCompra * 0.20; //20% de descompte
            preuFinal = totalCompra - descompte;
            System.out.println("Client VIP. Descompte aplicat: " + descompte + " €. Preu
                               final: " + preuFinal + " €.");
            return preuFinal;
        } else {
            System.out.println("Tipus de client no reconegut.");
            return totalCompra;
        }
    }
}
```

3.1. (2 punts) Reescriuiu el mètode *calculaPreuFinal* per tal de millorar-lo tot aplicant tècniques de refacció. Com a mínim, heu de realitzar 2 refaccions. Una d'aquestes modifica de manera important l'estructura del codi i puntua molt més que l'altra.

Nota: aplicareu les tècniques de refacció necessàries per a millorar aquest codi amb la finalitat de fer-lo més eficient i/o fàcilment llegible i/o fàcilment mantenible (modificable). No s'han de fer servir totes, només aquelles que considereu oportunes. De fet, amb dues n'hi ha prou. Exemples de tècniques de refacció són:

- Extreure una variable local.
- Extreure una constant.
- Extreure el mètode.
- Extreure una interfície.

No es tracta d'optimitzar matemàticament el càlcul, sinó de millorar la qualitat estructural del codi (mantenibilitat, llegibilitat, reutilització).

Escriviu el codi resultant al requadre següent. Poseu comentaris del tipus `//` en **color vermell** per indicar on heu aplicat tècniques de retroacció i quines heu aplicat. Així serà més difícil equivocar-se en la valoració de la vostra feina.

```
package refaccio;

public class GestorDescomptes {

    //Definicion de constantes para Los descuentos.
    private static final double DESCOMPTE_BASIC = 0.05;
    private static final double DESCOMPTE_PREMIUM1 = 0.10;
    private static final double DESCOMPTE_PREMIUM2 = 0.15;
    private static final double DESCOMPTE_VIP = 0.20;

    //Definicion de constantes para Los umbrales.
    private static final double UMBRAL_BAJO = 100;
    private static final double UMBRAL_MEDIO = 200;
    private static final double UMBRAL_ALTO = 500;

    //Definicion de constantes para Los tipos de clientes.
    private static final String CLIENTE_BASIC = "basic";
    private static final String CLIENTE_PREMIUM = "premium";
    private static final String CLIENTE_VIP = "vip";

    //Definicion de constante para mensaje de cliente no reconocido.
    private static final String CLIENTE_NO_RECONOCIDO = "Tipus de client no reconegut.";

    //Metodo principal para calcular el precio final despues de aplicar el descuento.
    public static double calculaPreuFinal(double totalCompra, String tipusClient)
    {
```



```

//Definir las variables y hacer el calculo del descuento y precio final.
double descompte = calcularDescuento(totalCompra, tipusClient);
double preuFinal = totalCompra - descompte;

//Llamada al metodo para mostrar el mensaje final y retornar el precio final.
mensaje(tipusClient, descompte, preuFinal);
return preuFinal;
}

//Metodo para calcular el descuento en formato switch para no repetir codigo y asi mejorar la legibilidad seleccionando solo el caso de cada cliente.
private static double calcularDescuento(double total, String tipus) {
    double descompte = 0;
    switch (tipus) {
        case CLIENTE_BASIC:
            if(total > UMBRAL_BAJO) descompte = total * DESCOMPTE_BASIC;
        case CLIENTE_PREMIUM:
            if (total > UMBRAL_BAJO) descompte = total * DESCOMPTE_PREMIUM1;
            if (total > UMBRAL_ALTO) descompte = total * DESCOMPTE_PREMIUM2;
        case CLIENTE_VIP:
            if(total > UMBRAL_MEDIO) descompte = total * DESCOMPTE_VIP;
        default:
            System.out.println(CLIENTE_NO_RECONOCIDO);
            return descompte;
    }
}

//Metodo para mostrar el mensaje final al usuario.
public static void mensaje(String text, double descompte, double preuFinal) {
    if (text == CLIENTE_BASIC || text == CLIENTE_PREMIUM || text == CLIENTE_VIP) {
        System.out.println("Client " + text + ". Descompte aplicat: " + descompte + " €. Preu final: " + preuFinal + " €.");
    }
    else {
        System.out.println(CLIENTE_NO_RECONOCIDO);
    }
}
}

```

3.2. (0,5 punts) Documenteu la classe anterior perquè es pugui generar la seva documentació amb Javadoc i enganxeu una captura de pantalla amb l'inici de la documentació que es genera per a aquesta classe i el codi resultant a continuació.

Observació: pot ser que en triar l'opció "Generate Javadoc..." us demani el camí del programa que genera la documentació automàticament. Si es així, heu de seleccionar el programa javadoc.exe de la carpeta bin de la instal·lació del JDK de Java.

Captura de l'inici documentació generada:

The screenshot shows the Javadoc output for the class `GestorDescomptes`. At the top, it lists the module (`refraccio`) and package (`refraccio`). The class is defined as `public class GestorDescomptes extends Object`. A brief description in Spanish states: "Clase que gestiona los descuentos aplicables a los clientes según su tipo y el total de compra. Incluye lógica para clientes Basic, Premium y VIP." It also includes version (1.0) and author (Achraf Ben Tamou) information.

Below the class information, there is a "Constructor Summary" section with a table:

Constructor	Description
<code>GestorDescomptes()</code>	

Next is the "Method Summary" section, which has tabs for "All Methods", "Static Methods", and "Concrete Methods". The "All Methods" tab is selected, showing a table:

Modifier and Type	Method	Description
static double	<code>calculaPreuFinal(double totalCompra, String tipusClient)</code>	Calcula el precio final de una compra aplicando el descuento correspondiente.
static void	<code>mensaje(String text, double descompte, double preuFinal)</code>	Muestra por consola el mensaje con el resultado de la operación.

At the bottom, it lists "Methods inherited from class java.lang.Object": `equals`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`.

4. (3,5 punts) Prova.

Tenint en compte el següent codi:

```
// Fitxer RiscTerratremol.java

package terratremol;

public class RiscTerratremol {

    public static String nivellRisc(double magnitud)
        throws MagnitudIncorrectaException{
        if(magnitud >= 0 && magnitud<= 10) {
            if (magnitud <= 2.0) return "No percebut";
            else if (magnitud <= 4.0) return "Lleu";
            else if (magnitud <= 6.0) return "Moderat";
            else if (magnitud <= 8.0) return "Fort";
            else return "Catastròfic";
        }
        else throw new MagnitudIncorrectaException("Valor de magnitud
incorrecte "+ magnitud +".");
    }
}

// Fitxer MagnitudIncorrectaException.java

package terratremol;

@SuppressWarnings("serial")
public class MagnitudIncorrectaException extends Exception {
    public LMagnitudIncorrectaException (String missatge) {
        super(missatge);
    }
}
```

El mètode *nivellRisc* de la classe *RiscTerratremol* retorna el nivell de risc en text associat a la magnitud d'un terratrèmol. Els nivells es defineixen segons el rang de valors de l'escala de Richter i, en cas que el mètode rebi un valor incorrecte (fora de l'interval permès), llença l'excepció *MagnitudIncorrectaException*.

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 11 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

4.1 (2,5 punts) Realitzeu el disseny dels casos de prova per comprovar el funcionament correcte del mètode *nivellRisc*, tenint en compte únicament el valor del paràmetre d'entrada *magnitud*, amb el codi que figura a l'enunciat.

El document DA2_M5_Exemples_Casos_Prova.pdf del material complementari de la Unitat 2 conté exemples complets.

Cal seguir els següents passos:

a) **(1 punt)** Dissenyeu les classes d'equivalència. Cal seguir els passos que s'indiquen al punt del material 1.3.2 (Disseny de les proves. Tipus de proves), apartat "Classes d'equivalència".

Per tant, els passos són:

- "Identificar les condicions, restriccions o continguts de les entrades i les sortides".
- "Identificar, a partir de les condicions, les classes d'equivalència de les entrades i les sortides".

Tots els valors d'una classe d'equivalència han de ser tractats igual.

Condición sobre magnitud	Válida o inválida	Equivalencia	Resultado esperado del método
$magnitud \leq 0$	Inválida	0.0 o -1.0	Lanza MagnitudIncorrectaException
$0 < magnitud < 2$	Válida	1.5	Devuelve texto "No percebut"
$2.0 \leq magnitud < 4.0$	Válida	3.5	Devuelve "Lieu"
$4.0 \leq magnitud < 6.0$	Válida	5.0	Devuelve texto "Moderat"
$6.0 \leq magnitud < 8.0$	Válida	7.0	Devuelve texto "Fort"
$8.0 \leq magnitud \leq 10.0$	Válida	9.0	Devuelve texto "Catastròfic"
$magnitud > 10.0$	Inválida	12.0	Lanza MagnitudIncorrectaException

b) **(1,5 punts)** Dissenyeu els casos de prova a partir de les classes d'equivalència seleccionades anteriorment. Tingueu en compte que:

- Com a mínim ha d'haver un representant de cada classe d'equivalència
- Han de cobrir els valors límit, el valor intermedi (si existeix; al cas dels intervals sense límit trivial pot agafar-se un altre valor qualsevol; si el resultat ha de ser enter i el valor intermedi surt amb decimals, podeu truncar o arrodonir) i els errors típics.
- Han de recórrer almenys una vegada cada camí independent (això últim està relacionat amb els apartats "Cobertura del flux de control" i "Complexitat ciclomàtica"). Per tant, caldrà:
 - Realitzar el diagrama de flux.
 - Calcular la complexitat ciclomàtica.
 - Escriure els camins independents.
 - Assegurar-se que els casos de prova cobreixen tots els camins; si detecteu que algun cas queda sense cobrir, cal afegir-hi més casos. Indiqueu quin(s) camí(ns) cobreix cada cas de prova.

4.2 (1 punt) Creeu un projecte amb les classes **nivellRisc** i **MagnitudIncorrectaException**. Executeu-lo, amb l'ajut de JUnit5, proves del mètode **nivellRisc(double magnitud)**, on cadascuna de les proves inclogui una crida a la funció amb els següents valors:

```
RiscTerratremol.nivellRisc(-1.0);
RiscTerratremol.nivellRisc(0.0);
RiscTerratremol.nivellRisc(1.5);
RiscTerratremol.nivellRisc(3.5);
RiscTerratremol.nivellRisc(5.0);
RiscTerratremol.nivellRisc(7.0);
RiscTerratremol.nivellRisc(9.0);
RiscTerratremol.nivellRisc(12.0);
```

Observacions (llegiu-les totes abans de començar):

- JUnit ja el teniu incorporat a Eclipse.
Heu de fer : File → new JUnit Test Case. Si no us apareix al menú, heu de triar el següent: File → new → Other → Java (apareix com una carpeta) → JUnit (apareix com una carpeta) → JUnit Test Case.
La biblioteca s'afegirà al projecte de manera automàtica, com es comenta més avall.
- Després, amb l'assistent que apareix, cal crear una classe de test (al material també indica com fer-ho). Ha de ser al mateix paquet que la classe que es vol provar i el seu nom ha de començar per Test (p.e. **TestNivellRisc**). També, l'assistent us permetrà afegir la biblioteca al projecte.
- En aquesta classe cal posar un mètode precedit per **@Test** per a cada prova que vulguem fer. Cal fer una prova per mètode, malgrat que, en general, és possible fer a cada mètode més d'una prova. Podeu posar tants mètodes com vulgueu. En executar les proves s'aniran executant els mètodes l'un darrere l'altre.
- Per controlar el resultat, cal utilitzar les anotacions o mètodes de JUnit que cregueu adients per fer cada prova. Podeu trobar informació al respecte a l'annex del primer apartat, “Disseny i realització de proves de programari”, del material. Cal tenir present que on posa “Declaracions” s'està referint a mètodes estàtics de la classe *org.junit.jupiter.api.Assertions*.
- Després, ja podeu executar aquesta classe com “JUnit Test”. Veureu que diu a quins mètodes ha anat bé la prova i a quins no.

Realitzeu una captura de pantalla de l'execució dels tests on es vegin els tests realitzats i enganxeu-la a continuació. A la captura s'han de veure els tests que s'executen i els resultats de l'execució.

	Codi: I71	Exercici d'avaluació contínua 3	Pàgina 13 de 15
	Versió: 03	DA2_M0487_EAC3_Enunciat_2526S1	Lliurament: 10/12/2025

```

package terratremol;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class TestNivellRisc {

    // CASO 1: Valor -1.0
    @Test
    void testNivellRiscNegativo() {
        assertThrows(MagnitudIncorrectaException.class, () -> {
            RiscTerratremol.nivellRisc(-1.0);
        });
    }

    // CASO 2: Valor 0.0
    @Test
    void testNivellRiscCero() {
        assertThrows(MagnitudIncorrectaException.class, () -> {
            RiscTerratremol.nivellRisc(0.0);
        });
    }

    // CASO 3: Valor 1.5
    @Test
    void testNivellRisc1_5() throws MagnitudIncorrectaException {
        assertEquals("No percebut", RiscTerratremol.nivellRisc(1.5));
    }

    // CASO 4: Valor 3.5
    @Test
    void testNivellRisc3_5() throws MagnitudIncorrectaException {
        assertEquals("Lleu", RiscTerratremol.nivellRisc(3.5));
    }

    // CASO 5: Valor 5.0
    @Test
    void testNivellRisc5_0() throws MagnitudIncorrectaException {
        assertEquals("Moderat", RiscTerratremol.nivellRisc(5.0));
    }

    // CASO 6: Valor 7.0
    @Test
    void testNivellRisc7_0() throws MagnitudIncorrectaException {
        assertEquals("Fort", RiscTerratremol.nivellRisc(7.0));
    }

    // CASO 7: Valor 9.0
    @Test
    void testNivellRisc9_0() throws MagnitudIncorrectaException {
        assertEquals("Catastrfic", RiscTerratremol.nivellRisc(9.0));
    }
}

```

```
// CASO 8: Valor 12.0
@Test
void testNivellRisc12_0() {
    assertThrows(MagnitudIncorrectaException.class, () -> {
        RiscTerratremol.nivellRisc(12.0);
    });
}
```

TestNivellRisc

Runs: 8/8 ✖ Errors: 0 ✖ Failures: 0

TestNivellRisc [Runner: JUnit 5] (0,025 s)

- ✓ testNivellRisc1_50 (0,015 s)
- ✓ testNivellRisc3_50 (0,000 s)
- ✓ testNivellRisc5_00 (0,000 s)
- ✓ testNivellRisc7_00 (0,000 s)
- ✓ testNivellRisc9_00 (0,000 s)
- ✓ testNivellRisc12_00 (0,005 s)
- ✓ testNivellRiscCero0 (0,001 s)
- ✓ testNivellRiscNegativo0 (0,001 s)